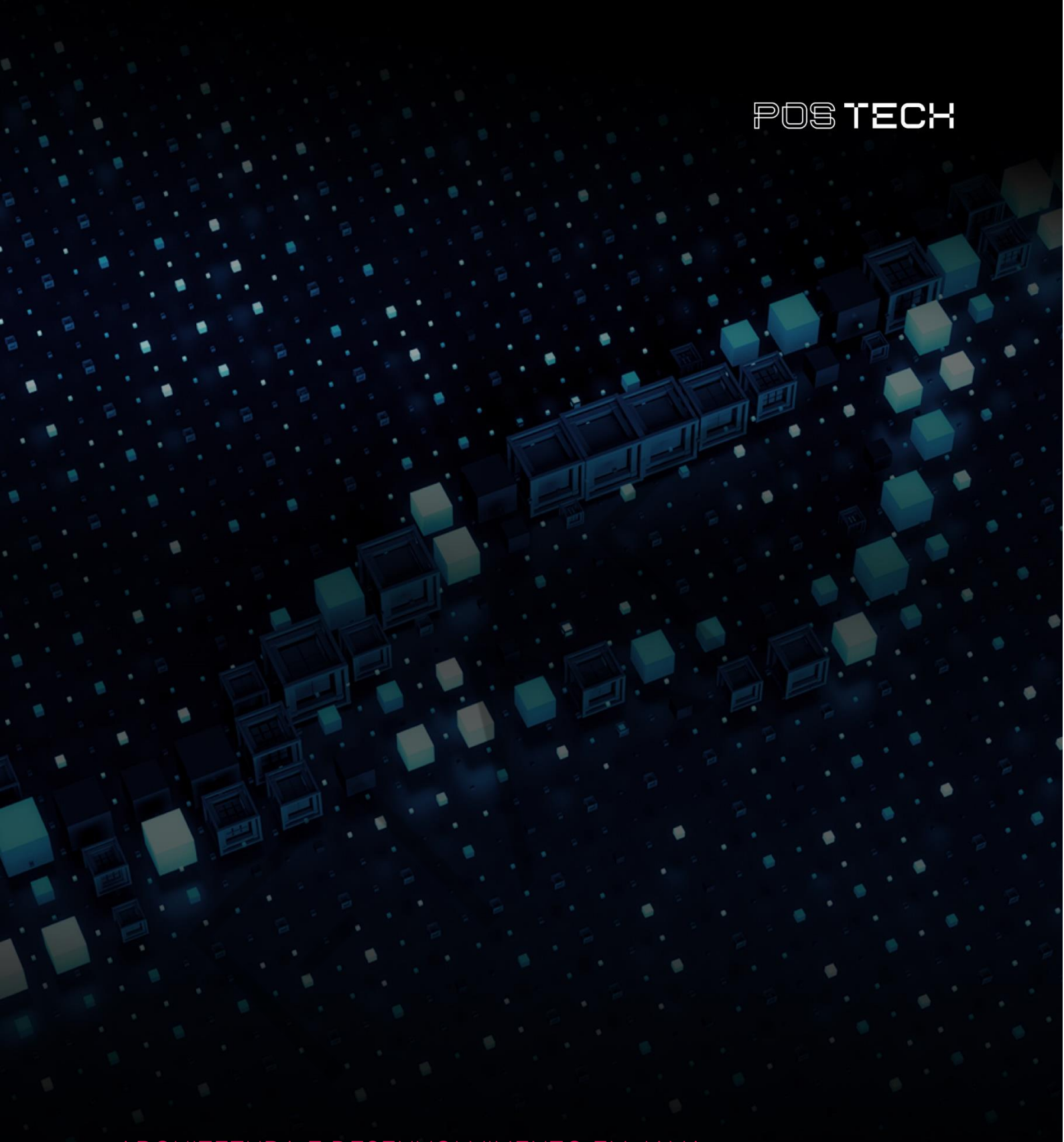


POS TECH

ARQUITETURA E DESENVOLVIMENTO EM JAVA

FASE 4 | AULA 03 -

PRIMEIROS PASSOS COM AZURE FUNCTIONS

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SUMÁRIO

O QUE VEM POR AÍ?.....3
HANDS ON 4
SAIBA MAIS.....5
O QUE VOCÊ VIU NESTA AULA? 11
REFERÊNCIAS.....12

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VEM POR AÍ?

Nesta aula, vamos entender como podemos usar a estrutura de função como serviço dentro de dois serviços de nuvem: o Google Cloud Platform e o Microsoft Azure, abordando as particularidades de cada um deles.

Também entenderemos que os desafios de implementação da arquitetura serverless têm muitos pontos em comum entre todos os provedores, como a questão do cold start e a restrição proposital de tempo de operação, para que as funções atuem como pontos de execução únicos e parte de uma solução maior.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

HANDS ON

Nesta aula, daremos os primeiros passos no uso do Google Cloud Platform e do Microsoft Azure.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SAIBA MAIS

Serverless na Google Cloud Platform (GCP)

Para implementar funções serverless na Google Cloud, usamos principalmente o serviço Cloud Run. Apesar de existirem outros, como Cloud Functions e AppEngine, o Cloud Run é o mais apropriado para implantação de funções como serviço por oferecer uma abordagem mais flexível, combinando a simplicidade do serverless com a portabilidade e o controle oferecidos pelos containers.

Detalhes técnicos

Seguindo uma prática que o GCP implementa na maioria dos seus produtos de cloud, fornecendo soluções gerenciais baseadas em ideias ou produtos open source, o Cloud Run funciona como um serviço de execução de containers, mas baseado em uma infraestrutura completamente gerenciada. Nos bastidores, ele utiliza o **Knative**, um projeto open source que roda sobre Kubernetes, mas esconde toda a complexidade de gestão de clusters e orquestração.

O ciclo de funcionamento do Cloud Run, no contexto de execução do java, pode ser descrito em algumas etapas:

Preparação e empacotamento da aplicação: a pessoa constrói sua aplicação de acordo com as regras de construção de aplicações Cloud Run e a publica usando a técnica que preferir, seja manualmente ou com alguma facilidade de deploy proporcionada pelo framework em uso.

Deploy: internamente, o código é isolado dentro de uma imagem que é enviada para o Artifact Registry e, em seguida, implantada no Cloud Run.

Resposta a gatilho de execução da função: como padrão em todas as arquiteturas de cloud, o Cloud Run expõe automaticamente um endpoint HTTP/HTTPS seguro interno, para que os diferentes serviços possam acessar esta função. A partir desse momento, inclusive, a aplicação começa a escutar requisições HTTP sem que a pessoa desenvolvedora precise configurar load balancers, gateways ou certificados SSL.

Modelo de execução: cada requisição é tratada de forma isolada ou compartilhada, dependendo da configuração de concurrency (concorrência por

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

instância). Por padrão, cada instância pode atender até 80 requisições simultâneas, de acordo com uma configuração. Mas a construção da função deve ser pensada no modelo stateless, para evitar dependências que podem não estar disponíveis.

Um ponto interessante na Cloud Run é que não estamos limitados apenas a um processo de empacotamento padrão de acordo com a linguagem selecionada. É possível, também, construir manualmente um container e publicar para execução no Cloud Run. Então, seu grande diferencial em relação a outros serviços semelhantes, em outros provedores de nuvem, é a possibilidade de escolher como o empacotamento da aplicação será feita. Por exemplo, podemos usar o serviço Cloud Build para ajudar na construção das aplicações, caso decida pelo uso de containers diretamente.

Isso também facilita no processo de ciclo de vida da aplicação, uma vez que utilizar containers torna o ambiente altamente portátil. A mesma imagem que roda no Cloud Run pode ser executada localmente, em outro provedor cloud ou até em Kubernetes gerenciado (como GKE). Um exemplo de código:

```
import com.google.cloud.functions.HttpFunction;
import com.google.cloud.functions.HttpRequest;
import com.google.cloud.functions.HttpResponse;

import java.io.IOException;
import java.io.PrintWriter;

public class GreetingFunction implements HttpFunction {
    @Override
    public void service(HttpRequest request, HttpResponse
response) throws IOException {
        response.setContentType("text/plain");
        PrintWriter writer = response.getWriter();
        writer.write("Você está vendo uma função rodando no
Cloud Run!");
    }
}
```

Código-fonte 1 — Exemplo 1

Fonte: elaborado pelo autor (2025)

No caso acima, temos uma implementação extremamente simples: uma classe implementa **HttpFunction** e, por consequência, o método **service**; ao ser publicada na Cloud Run, ela é executada ao receber uma chamada de um gatilho http.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Integrações

Implantar a solução considerando a integração com os serviços da Google Cloud é importante para que ela funcione corretamente. Por exemplo, entender como funciona a atribuição de papéis (roles) e direitos de acesso a outros componentes é, ao mesmo tempo, um desafio para a implementação técnica da solução e uma oportunidade de desenhar corretamente a aplicação, considerando que os serviços disponíveis (como Cloud Storage e Workflows) podem ser usados como origem e destino de diversos gatilhos.

Desafios de implementação

Na implementação de funções com o Cloud Run, também temos que lidar com desafios, alguns semelhantes ao que encontramos em outros serviços de nuvem.

Cold Start: quando não há nenhuma instância rodando, a primeira requisição após um período de inatividade pode ter uma latência elevada (em média de 1 a 3 segundos, podendo ser mais demorado para aplicações Java, devido ao tempo de inicialização da JVM e do framework). Técnicas como graalVM nativo, uso de frameworks otimizados (Micronaut e Quarkus) ou manter instâncias mínimas ativas (funcionalidade paga) podem mitigar esse problema.

Limitações de tempo e recursos: cada requisição possui um tempo máximo de execução de 60 minutos, além de ter um limite de CPU e memória (até 32 CPUs e 128 GB de RAM por instância atualmente). Ainda assim, funções de longo tempo de execução ou o consumo de uma quantidade muito grande de recursos incorre em um aumento do valor a ser pago por execução. Para workloads mais longos ou de processamento intensivo, é necessário avaliar alternativas como o Cloud Run Jobs, Workflows ou GKE.

Gerenciamento de estado: como definido pela arquitetura, os componentes de função como serviço são, por definição, stateless (sem estado). Qualquer dado que precise persistir entre requisições deve ser armazenado externamente em serviços como Cloud Storage, Firestore, Cloud SQL ou Memorystore.

Conexões com bancos de dados: a escalabilidade elástica pode gerar explosão de conexões simultâneas com bancos de dados tradicionais (ex.: PostgreSQL, MySQL), que possuem limites de conexão, fazendo com que o acesso a este recurso se torne um gargalo de processamento, aumentando o tempo de uso

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

da função. É importante pensar em soluções de arquitetura, que podem ser, por exemplo, a limitação de funções simultâneas, uso de cache ou técnicas de CQRS, utilização de connection pools gerenciados (como o Cloud SQL Proxy ou o Cloud SQL Auth Proxy Connector) e avaliar o uso de bancos serverless (ex.: Firestore ou AlloyDB Omni).

Observabilidade e debug: em ambientes altamente dinâmicos e efêmeros, o monitoramento torna-se muito mais complexo, porque o rastreamento de um problema deve levar em conta o processamento em todos os componentes envolvidos, sejam funções ou não. É essencial implementar corretamente logs estruturados e centralizados, métricas e tracing distribuído (Stackdriver, OpenTelemetry e Cloud Trace).

Modelagem de aplicações: arquiteturas monolíticas podem ser migradas para Cloud Run, mas é possível que não aproveitem completamente os benefícios do serverless, podendo inclusive aumentar o custo. A divisão da aplicação em microsserviços ou funções bem definidas, seja em Cloud Run pura ou publicação em containers, permite maior elasticidade e manutenção, mas traz desafios de orquestração, versionamento, governança e segurança.

Possíveis soluções

A primeira é a utilização de frameworks modernos, que foram pensados para uso com containers, como o Quarkus ou o Micronaut. Caso não seja possível alterar o framework atual, o uso de GraalVM (com GraalVM Native Image) para compilação nativa pode ajudar a melhorar o desempenho de aplicações desenvolvidas usando Spring.

Para centralização de logs, é possível usar o Cloud Logging, gravando os logs de aplicação diretamente neste serviço. Essa solução precisa levar em conta, também, a quantidade de logs necessária, que deve ser adequada para não gerar poluição de entradas, aumentando o custo de armazenamento dos logs e dificultando o rastreamento, ou entradas insuficientes, que não criam condições de entendimento claro no rastreamento dos problemas.

Usando Quarkus para a solução

O Quarkus disponibiliza dois componentes para o uso de funções na Google Cloud.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

- **google-cloud-functions:** essa extensão permite criar, de forma simples, funções que podem ser implementadas diretamente para Cloud Run e, via anotações, conectar gatilhos a cada uma das funções. Usando esta extensão, não é necessário criar manualmente os pacotes de deploy, o próprio comando de deploy do Quarkus CLI faz a implantação.

- **google cloud service extensions:** ao usar o code.quarkus.io para criar sua aplicação, procurando por google cloud, temos diversas extensões para os serviços da Google Cloud. Para consumir os serviços, precisamos usar a extensão relacionada a ele. Exemplo: para usar o serviço de armazenamento de arquivos (Google Cloud Storage), a extensão é quarkus-google-cloud-storage.

SERVERLESS NA AZURE

Assim como para os outros provedores de nuvem, é possível fazer uso da estrutura de funções como serviço na nuvem da Microsoft. Aqui, todas vantagens e desafios se repetem: temos a possibilidade de usar as Azure Functions com java, aproveitando as bibliotecas fornecidas pela Microsoft, e preparando o código para rodar de acordo com as regras predefinidas.

A principal diferença, aqui, é a possibilidade de configurar dados dos gatilhos diretamente na annotation do método que será a “entrada” (entrypoint) para a função, como podemos ver abaixo:

```
public class HelloSpringFunction {
    private static final ApplicationContext context =
        new
AnnotationConfigApplicationContext(AppConfig.class);

    private     final     GreetingService     service     =
context.getBean(GreetingService.class);

    @FunctionName("helloSpring")
    public HttpResponseMessage run(
        @HttpTrigger(
            name = "req",
            methods = {HttpMethod.GET, HttpMethod.POST},
            authLevel = AuthorizationLevel.ANONYMOUS)
        HttpRequestMessage<Optional<String>> request,
        final ExecutionContext context) {
```

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

```
            String                      name                      =
request.getQueryParameters().get("name");
            String message = service.greet(name);

            return
request.createResponseBuilder(HttpStatus.OK)
                .body(message)
                .build();
        }
    }
```

Código-fonte 2 — Exemplo 2

Fonte: elaborado pelo autor (2025)

O HttpTrigger define o tipo de gatilho esperado (request HTTP) e quais os métodos disponíveis, que se torna o arquétipo do objeto esperado em request. O objeto de context, do tipo ExecutionContext, informa os dados da execução e construímos a resposta usando os métodos de ResponseBuilder do objeto request.

Com as bibliotecas da Azure, temos também facilidades como deploy com configurações específicas para o Azure, fazendo com que o processo esteja integrado diretamente na execução de comandos do maven.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Vimos como usar a estrutura de função como serviço no Google Cloud Platform e no Microsoft Azure, além dos desafios de implementação da arquitetura serverless nesses provedores.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

REFERÊNCIAS

GOOGLE Cloud. **Modelo de arquitetura do Google Cloud**. Disponível em https://cloud.google.com/architecture/framework?hl=pt-br . Acesso em: 07 jul. 2025.

INDRASIRI, K; SUHOTHAYAN, S. **Design Patterns for Cloud Native Applications**. [S.l.]: O'Reilly, 2021.

MICROSOFT. **Azure Architecture Center**. Disponível em https://learn.microsoft.com/en-us/azure/architecture. Acesso em: 07 jul. 2025.

NEWMAN, S. **Building Microservices**: Designing Fine-Grained Systems. [S.l.]: O'Reilly, 2021.

FIAP

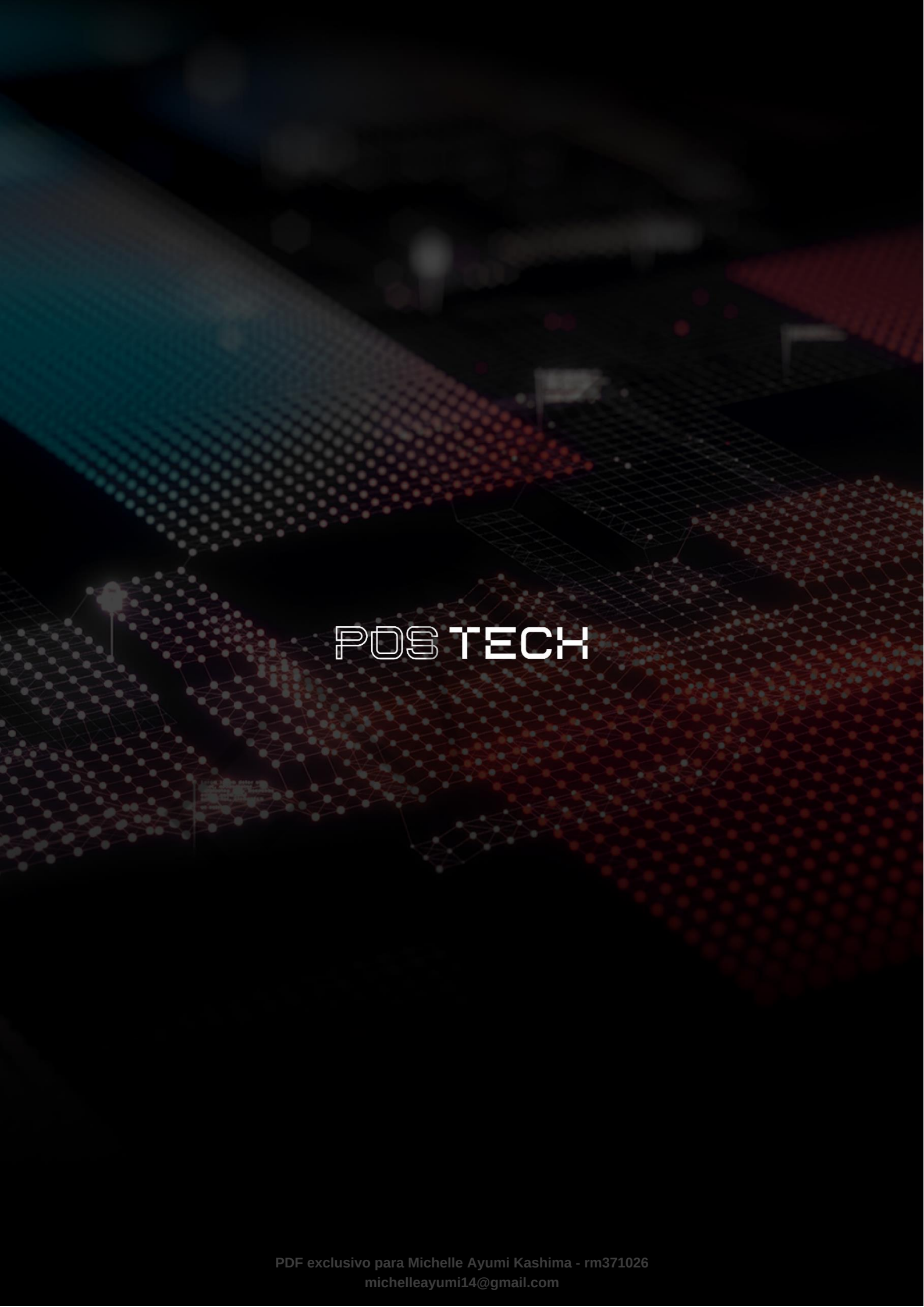
PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

PALAVRAS-CHAVE

Serverless. Google Cloud. Microsoft Azure.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com



PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com